

LLM Notes

2026 Spring

Day 18: Autograd Saved Tensors 与 Operator Fusion

CS336 Systems

主讲: Aaron

Fudan University

今日摘要

2026.7.20 Autograd 为了在 backward 中计算梯度, 会保存 forward 中的一部分中间 tensor。通过 `saved_tensors_hooks` 可以观察这些 tensor。实验中, 普通 RMSNorm 保存约 80.03 MiB, 而经过 `torch.compile` 后只保存约 20.01 MiB, 说明编译器可以通过融合、消除中间结果和适当重计算降低 activation memory。

18.1 为什么 Backward 需要保存 Tensor

假设:

$$y = f(x, W).$$

Backward 需要计算:

$$\frac{\partial L}{\partial x} \quad \text{和} \quad \frac{\partial L}{\partial W}.$$

这些梯度通常依赖 forward 中的输入或中间结果, 因此 autograd 会将部分 tensor 保存到计算图中, 等 backward 使用完成后再释放。

这些 tensor 称为:

- saved tensors;
- forward residuals;
- activations saved for backward。

18.2 Saved Tensor Hooks

PyTorch 提供：

```
torch.autograd.graph.saved_tensors_hooks(  
    pack_hook,  
    unpack_hook,  
)
```

当 autograd 准备保存一个 tensor 时，会调用 `pack_hook`；当 backward 需要重新取出它时，会调用 `unpack_hook`。

这里没有压缩或移动 tensor，只记录其 shape、dtype 和 logical size。

备注

`numel() * element_size()` 得到的是 tensor 的 logical size。如果多个 view 共享同一块 storage，将它们分别相加可能重复计算底层物理显存，所以该结果不能直接等同于真实 CUDA peak memory。

18.3 为什么实验输入需要 Requires Grad

独立测试一个模块时，我们使用：

```
x = torch.randn(  
    batch_size,  
    sequence_length,  
    d_model,  
    device=device,  
    requires_grad=True,  
)
```

设置 `requires_grad=True` 表示 backward 还需要计算：

$$\frac{\partial L}{\partial x}.$$

在完整 Transformer 中，一个 block 的输入通常是前一层的输出，它会自动带有 gradient history。独立测试单个 block 时，手动开启 `requires_grad` 可以模拟这一情况。

它并不表示训练数据本身通常需要梯度，也不表示 optimizer 会更新输入。

18.4 RMSNorm 实验

RMSNorm 的主要计算可以写为：

$$r = \text{rsqrt} \left(\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon \right),$$

$$y_i = x_i r w_i.$$

实验输入形状为：

$$x \in \mathbb{R}^{4 \times 512 \times 2560},$$

使用 FP32，因此一个完整输入 tensor 大小为：

$$4 \times 512 \times 2560 \times 4 = 20 \text{ MiB}.$$

18.4.1 未编译版本

普通 eager RMSNorm 保存了多个中间 tensor，包括：

- RMSNorm 输入；
- reduction 和 `rsqrt` 相关结果；
- broadcast 后的 view；
- 逐元素乘法所需的输入。

记录得到的 saved tensor logical size 约为：

$$M_{\text{eager}} = 80.03 \text{ MiB}.$$

其中多个 shape 接近输入大小，因此总量约为输入 tensor 的四倍。

18.4.2 编译版本

随后使用：

```
compiled_rms_norm = torch.compile(
    rms_norm,
    fullgraph=True,
)
```

编译后的正式测量只保存了三个主要 tensor:

Shape	Logical size
(4, 512, 2560)	20.00 MiB
(2560)	约 0.01 MiB
(2048, 1, 1)	约 0.01 MiB

总 logical size 为:

$$M_{\text{compiled}} = 20.01 \text{ MiB.}$$

18.5 为什么 Compile 可以减少 Saved Tensors

Eager execution 逐个执行 Python/PyTorch operator。不同 operator 通常会产生各自的输出 tensor，而 autograd 也会按照这些 operator 的 backward 需求保存中间结果。

编译时，PyTorch 可以联合分析完整的 forward 和 backward graph，并进行：

- operator fusion;
- 消除不必要的中间 tensor;
- 缩短临时 tensor 的生命周期;
- 只保存 backward 真正需要的信息;
- 在适当位置重新计算便宜的中间结果。

例如，若 eager 模式依次执行：

$$x^2 \longrightarrow \text{mean} \longrightarrow \text{rsqrt} \longrightarrow x \cdot r,$$

每一步都可能产生独立 tensor。编译器可以把多个逐元素和 reduction 操作组合成更少的 kernel，并避免将所有中间结果写回显存。

分析

saved tensor 的减少不应只归因于单一的 kernel fusion。torch.compile 还会通过 AOTAutograd 联合优化 forward 和 backward，决定哪些值需要保存、哪些值可以消除或重新计算。